



Smart Refrigerator Door Alert (เตือนลิ้นปิดตู้เย็น)

จัดทำโดย

กัญญาณัฐ	สุดเชยชม	1660700974
จตุพร	สุบิน	1660600188
อาภรณ์ภัทร์	ปานช้าง	1660701473
กษิรา	สาริมานนท์	1660701234

อาจารย์ผู้สอน

อาจารย์ เลอศักดิ์ ลิ้มวิวัฒน์กุล

หลักสูตรวิทยาศาสตรบัณฑิต

ภาคการศึกษาที่ 2 ปีการศึกษา 2568

เอกสารนี้เป็นส่วนหนึ่งของวิชา CS423 Internet of Things and Applications

ภาควิชาวิทยาการคอมพิวเตอร์ คณะเทคโนโลยีสารสนเทศและนวัตกรรม

สารบัญ

บทที่ 1

- บทนำ

บทที่ 2

- อุปกรณ์ที่ใช้และวิธีการต่อวงจร

2.1) อุปกรณ์ที่ใช้

1. ESP32 DevKit V1 (ESP-WROOM-32)
2. LDR Photosensitive Resistor Module(LM393)
3. Buzzer
4. LED
5. Button
6. Ultrasonic Sensor(HC-SR04)
7. Resistor 220 Ω
8. Breadboard
9. Jumper Wires

2.2) Schematic Diagram

2.3) Wiring Diagram

2.4) วิธีการต่อวงจร

1. การต่อวงจร LED (Blink Test)
2. การต่อวงจร Button + LED + Buzzer
3. การต่อวงจร Ultrasonic Sensor (HC-SR04)
4. การต่อวงจร LDR Sensor

บทที่ 3

- Testing code upload
 1. ขาอุปกรณ์ทั้งหมดที่ใช้
 2. ทดสอบการอัปโหลดโค้ด ESP32 และการทำงานของ LED
 3. ทดสอบ Button + LED + Buzzer
 4. ทดสอบ Ultrasonic Sensor (HC-SR04)
 5. ตรวจสอบแสงจาก LDR Sensor
 6. ได้รวมระบบ (Final System)

บทที่ 4

- สรุปการทดลองทั้งหมด

บทที่ 1

บทนำ

ปัจจุบันปัญหาการ ลืมปิดประตูเย็น เป็นสิ่งที่พบได้บ่อยในบ้านเรือน หอพัก หรือสำนักงาน ซึ่งอาจทำให้ความเย็นภายในตู้เย็นรั่วไหล ส่งผลให้ตู้เย็นทำงานหนักขึ้น สิ้นเปลืองพลังงานไฟฟ้า และอาจทำให้อาหารภายในตู้เย็นเสียได้ นอกจากนี้ยังอาจส่งผลกระทบต่ออายุการใช้งานของตู้เย็นในระยะยาว ดังนั้นการมีระบบที่สามารถตรวจสอบและแจ้งเตือนเมื่อประตูตู้เย็นเปิดทิ้งไว้จึงเป็นสิ่งที่มีความประโยชน์อย่างมาก

โครงการ Smart Refrigerator Door Alert (ระบบตรวจสอบและแจ้งเตือนการปิดตู้เย็น) จึงถูกพัฒนาขึ้นเพื่อช่วยตรวจสอบสถานะของประตูตู้เย็นแบบอัตโนมัติ พร้อมทั้งแจ้งเตือนเมื่อประตูตู้เย็นถูกเปิดทิ้งไว้นานเกินกำหนด โดยใช้เซ็นเซอร์และไมโครคอนโทรลเลอร์ในการควบคุมการทำงานของระบบ

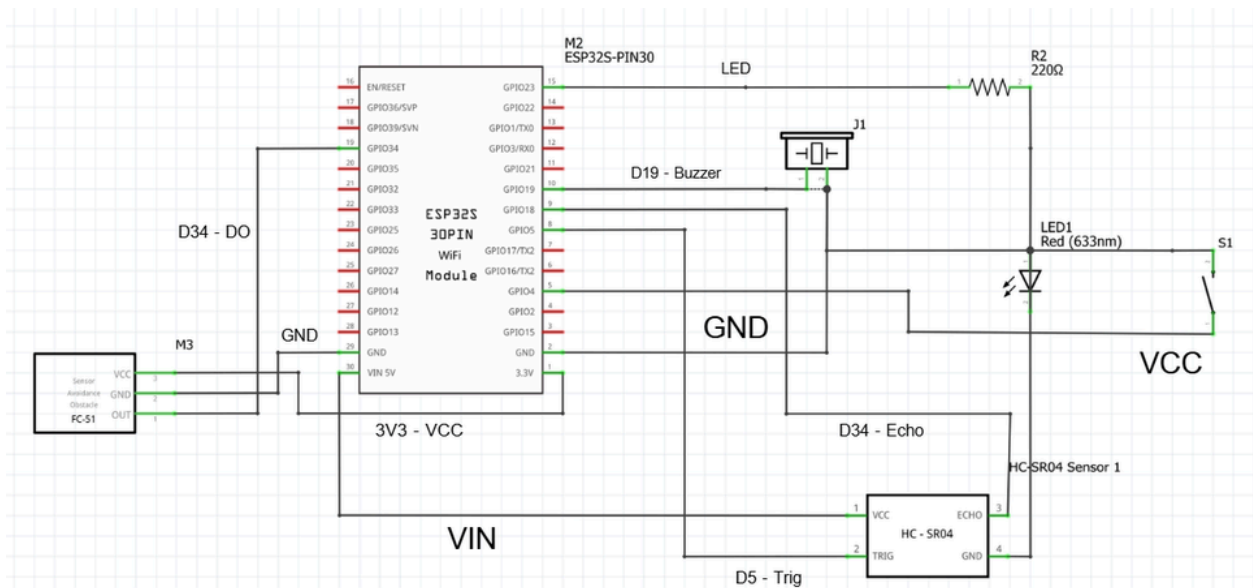
บทที่ 2

อุปกรณ์ที่ใช้และวิธีการต่อวงจร

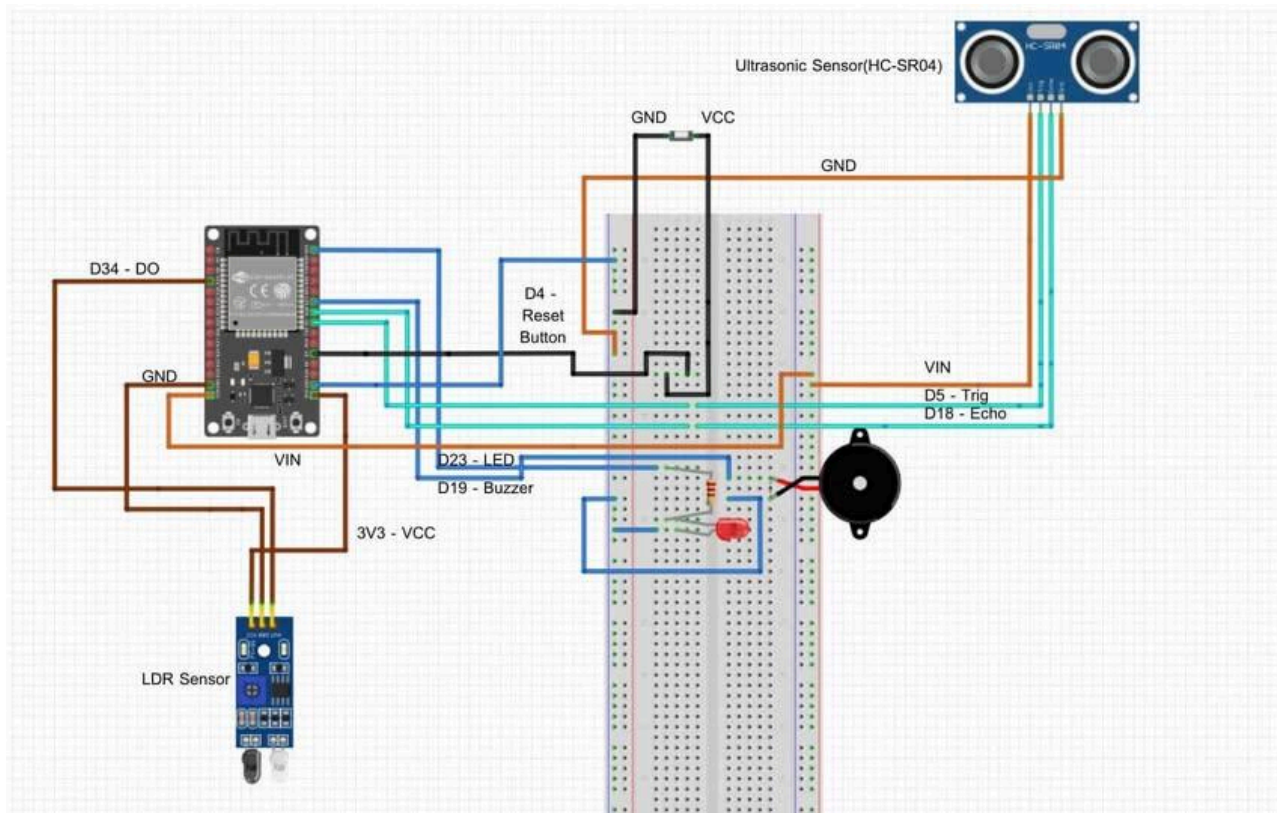
2.1 อุปกรณ์ที่ใช้

1. ESP32 DevKit V1 (ESP-WROOM-32)
2. LDR Photosensitive Resistor Module(LM393)
3. Buzzer
4. LED Light Sensor Module
5. Button
6. Ultrasonic Sensor(HC-SR04)

2.2 Schematic Diagram



2.3 Wiring Diagram



2.4 วิธีการต่อวงจร

1. การต่อวงจร LED (Blink Test)

อุปกรณ์

- LED
- ตัวต้านทาน 220Ω
- ESP32
- Breadboard

การต่อวงจร

- ขา Anode (+) ของ LED ต่อกับ ตัวต้านทาน GPIO 23
- ขา Cathode (-) ของ LED ต่อกับ GND ของ ESP32

2. การต่อวงจร Button + LED + Buzzer

อุปกรณ์

- Push Button
- LED
- Buzzer
- ตัวต้านทาน 220Ω
- ESP32

การต่อวงจร Button

- ขาหนึ่งของ Button ต่อกับ GPIO 4
- อีกขาหนึ่งของ Button ต่อกับ GND

การต่อวงจร LED

- LED Anode (+) ต่อกับ ผ่านตัวต้านทาน $\rightarrow$ GPIO 23
- LED Cathode (-) ต่อกับ GND

การต่อวงจร Buzzer

- Buzzer (+) ต่อกับ GPIO 19
- Buzzer (-) ต่อกับ GND

3. การต่อวงจร Ultrasonic Sensor (HC-SR04)

อุปกรณ์

- Ultrasonic Sensor HC-SR04
- ESP32

HC-SR04 มี 4 ขา

- VCC ต่อกับ 5V (VIN) ของ ESP32
- GND ต่อกับ

- Trig ต่อกับ GPIO 5
- Echo ต่อกับ GPIO 18

4. การต่อวงจร LDR Sensor

อุปกรณ์

- LDR
- ตัวต้านทาน $10k\Omega$
- ESP32

LDR ต้องต่อแบบ Voltage Divider

การต่อวงจร

- ขาหนึ่งของ LDR ต่อกับ 3.3V
- อีกขาของ LDR ต่อกับ GPIO 34
- ตัวต้านทาน $10k\Omega$ ต่อกับ ระหว่าง GPIO 34 กับ GND

บทที่ 3

Testing code upload

1. ขาอุปกรณ์ทั้งหมดที่ใช้

LDR Sensor	GPIO 34
Buzzer	GPIO 19
Ultrasonic TRIG	GPIO 5
Ultrasonic ECHO	GPIO 18
LED	GPIO 23
Button	GPIO 4

2. ทดสอบการอัปโหลดโค้ด ESP32 และการทำงานของ LED

```
const int ledPin = 23;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  delay(1000);
  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

ถ้า LED กระพริบ แสดงว่า

- ESP32 ทำงานปกติ
- การอัปโหลดโค้ดสำเร็จ
- ขา GPIO สามารถส่งสัญญาณได้
- การต่อ LED ถูกต้อง

3. ทดสอบ Button + LED + Buzzer

```
#define BUTTON_PIN 4
#define BUZZER_PIN 19
#define LED_PIN 23

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == 0) {
    digitalWrite(LED_PIN, HIGH);
    tone(BUZZER_PIN, 2000);
    Serial.println("Key Pressed! (LED ON & BUZZING)");
  }
  else {
    digitalWrite(LED_PIN, LOW);
    noTone(BUZZER_PIN);
  }

  delay(50);
}
```

เมื่อมีการกดปุ่ม

- LED จะติด
- Buzzer จะส่งเสียงที่ความถี่ 2000 Hz
- Serial Monitor จะแสดงข้อความ Key Pressed! (LED ON & BUZZING)

4. ทดสอบ Ultrasonic Sensor (HC-SR04)

```
#define TRIG_PIN 5
#define ECHO_PIN 18
#define LED_PIN 23

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);
  float distance = duration * 0.034 / 2;

  if (distance > 0) {
    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");

    digitalWrite(LED_PIN, HIGH);
    delay(50);
    digitalWrite(LED_PIN, LOW);
```

```
}  
else {  
  Serial.println("Error: No Sensor Detected!");  
}  
delay(500);  
}
```

เปิด **Serial Monitor**

- เอามือหรือวัตถุเข้าใกล้เซ็นเซอร์
- ผลลัพธ์ที่ถูกต้อง

-เมื่อวัตถุเข้าใกล้ ค่าระยะ ลดลง

-เมื่อวัตถุห่างออก ค่าระยะ เพิ่มขึ้น

5. ตรวจสอบแสงจาก LDR

```
#define LDR_PIN 34
#define BUZZER_PIN 19

void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {

  int lightValue = analogRead(LDR_PIN);

  if (lightValue > 3000) {
    Serial.println("Dark - Buzzer OFF");
    noTone(BUZZER_PIN);
  }
  else {
    Serial.println("Bright - Buzzer ON");
    tone(BUZZER_PIN, 4000);
  }

  delay(100);
}
```

ใช้มือบังเซ็นเซอร์ LDR

- ผลลัพธ์ที่ถูกต้อง

-เมื่อ เอามือบัง ค่าเพิ่มขึ้น

-เมื่อ เปิดไฟหรือส่องไฟ ค่าลดลง

6. โค้ดรวมทดสอบอุปกรณ์ ESP32

```
#define LDR_PIN 34
#define BUZZER_PIN 19
#define LED_PIN 23
#define BUTTON_PIN 4

#define TRIG_PIN 5
#define ECHO_PIN 18

unsigned long lightStart = 0;
unsigned long distanceStart = 0;
unsigned long blinkTimer = 0;

bool lightActive = false;
bool distanceActive = false;
bool alarmState = false;
bool systemLocked = false;
bool printed = false;

float lastDistance = 0;

float getDistance() {

    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH);
    return duration * 0.034 / 2;
}

void setup() {
```

```
Serial.begin(115200);

pinMode(LED_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT_PULLUP);

pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);
}

void loop() {

  int lightValue = analogRead(LDR_PIN);
  float distance = getDistance();

  // ----- DARK MODE -----
  if (lightValue > 3000) {

    alarmState = false;
    lightActive = false;
    distanceActive = false;

    digitalWrite(LED_PIN, LOW);
    noTone(BUZZER_PIN);

    systemLocked = false;
    printed = false;

    return;
  }

  // ----- BUTTON RESET -----
  if (digitalRead(BUTTON_PIN) == LOW) {

    alarmState = false;
    systemLocked = true;

    digitalWrite(LED_PIN, LOW);
    noTone(BUZZER_PIN);
```



```

    printed = false;

    return;
}

if (systemLocked) {
    return;
}

// ----- LIGHT DETECT -----
if (!lightActive) {

    lightActive = true;
    lightStart = millis();
}

digitalWrite(LED_PIN, HIGH);

// ----- DISTANCE CHANGE -----
if (abs(distance - lastDistance) > 1) {

    if (!distanceActive) {

        distanceActive = true;
        distanceStart = millis();
    }
}

lastDistance = distance;

unsigned long lightTime = (millis() - lightStart) / 1000;
unsigned long distanceTime = (millis() - distanceStart) / 1000;

// ----- ALARM CONDITION -----
if (lightActive && distanceActive &&
    lightTime >= 5 && distanceTime >= 5) {

    alarmState = true;

```

```

}

// ----- ALARM MODE -----
if (alarmState) {

    if (!printed) {
        Serial.println("ALERT!");
        printed = true;
    }

    if (millis() - blinkTimer > 300) {

        digitalWrite(LED_PIN, !digitalRead(LED_PIN));
        blinkTimer = millis();
    }

    tone(BUZZER_PIN, 4000);
}

delay(100);
}

```

- ระบบอ่านค่า LDR
- ถ้ามีด LED และ Buzzer ไม่ทำงาน
- ถ้าสว่าง LED ติด
- ถ้ามีแสงและระยะเปลี่ยนและเกิดเกิน 5 วินาที
- LED กระพริบ + Buzzer ดัง
- กดปุ่ม = รีเซ็ตระบบ LED ไม่ติด , Buzzer ไม่ดัง กดได้ก็ต่อเมื่อเกิน 5 วิ
- ต้องมีติดก่อนแล้วสว่างใหม่ ระบบจึงเริ่มทำงานอีกครั้ง

บทที่ 4

สรุปการทดลองทั้งหมด

- LED ESP32 ส่ง Output ได้
- Button ESP32 อ่าน Input ได้
- Buzzer Output เสียงทำงาน
- Ultrasonic วัดระยะได้
- LDR อ่านค่าความสว่างได้